

ECHO ORB

Puzzle-Adventure Unity 2D Game with
Level Editor



BSc (Hons) in Applied Computing
Sinead Cleary (20102299)
Supervisor: Brendan Lyng
Target platform: Windows PC

Contents

1. Introduction.....	2
2. Final Game Overview.....	3
3. Implemented Design and Game Systems	6
3.1 Core Gameplay Loop	6
3.1.1 Collect Orbs	6
3.1.2 Power Echo Holder	7
3.1.3 Activate All Powered Holders	7
3.1.4 Unlock Further Exploration.....	7
3.2 Player Mechanics.....	8
3.3 AI Systems.....	9
3.3.1 Simple Creature.....	9
3.3.2 Disguised Creature.....	10
3.3.3 Mischievous Creature.....	10
3.3.4 Crazy Creature	11
3.4 Level Editor.....	12
3.4.1 User Interface	12
3.4.2 Save and load.....	13
3.4.3 Level Validation.....	13
3.5 Level Design	14
4. Production Process and Project Management.....	16
5. Playtesting and Iteration.....	20
6. Technical Challenges and Solutions	24
6.1 Challenge 1 - Mischievous Creature Unintended Behaviour.....	24
6.2 Challenge 2 - Level Data Handling.....	24
6.3 Challenge 3 - Editor Layers.....	24
7. Refection and Learning	26
8. Future Work and Unimplemented Features	27
9. Conclusion.....	28
10. References.....	29
11. Credits	30
Music	30
Sound effects.....	30
Other	30

1. Introduction

Echo Orb is a 2D puzzle-adventure game with custom level editor, which was not only used to create the game's base levels, but is also included in-game for custom level creation. The game is made for PC and was created using Unity 6. The goal of the core gameplay is to complete all the base game levels. Each level is completed by traversing the dungeon environment and reaching the endpoint. Traversal comes with many challenges, however, as the mysterious dungeon contains strange creatures, deadly traps, and walls that are seemingly not always there. The player must use the Echo system, which controls the environment and its interconnected systems, to navigate through the levels. The built-in level editor allows players to experiment with their own puzzles, making use of the existing gameplay elements.

The aims of this project are to combine the knowledge I have gained over the course of my studies to create a polished game which demonstrates a complete gameplay loop, to use agile methodology to organise and manage project development, to create and use original assets where possible, and to create an overall experience that is enjoyable and engaging for the user.

While the project was ambitious in scope, it was necessary to remain realistic in order to account for limited development time in order to meet project deadlines, as well as time required to address gaps in my knowledge; the workload that comes with solo development, including programming, design and asset creation; and having a limited number of playtesters to provide feedback. With these constraints in mind, I prioritised core gameplay over extra features.

This report aims to detail the finished game, demonstrate technical complexity and innovation, and reflect on the development process and what I have learned throughout the course of this project. The report is structured as follows: Section 2 provides the final game overview, replacing a traditional "user manual", Section 3 details implemented design and gameplay systems, Section 4 looks at the overall production process and methodology used in the development of the project, Section 5 describes the playtesting methodology used and lists playtesting feedback, Section 6 describes some of the key technical challenges faced during the development of the project, Section 7 reflects on the skills I have learned and the knowledge I have developed, and Section 8 discusses potential future work. Finally, the report conclusion includes the final outcomes of the project and the key takeaways from the development process.

2. Final Game Overview

Echo Orb's puzzle mechanics aim to challenge the player in a way that is fun, and to make them think. Often a sequence of moves must be completed in the correct order to progress, requiring experimentation and careful planning.

The central Echo system drives the core gameplay. It works both with and against the player, serving both as a puzzle solving tool and as a part of the puzzle itself. The Echo system is made up of Echo Orbs and Echo Holders. Echo Orbs are collectable items which can be found spread across the levels and are used to power Echo Holders. Echo Holders are stationary interactable elements that control the dungeon environment and its mechanisms. There are multiple different types of Echo Holders, each with their own unique effects, such as toggling invisible walls (known as Ghost Walls), firing cannons, or teleporting the player. The player can perform an activation from anywhere in the level, triggering the effects of all powered Echo Holders simultaneously. Figure 1 shows the player performing an activation and the resulting effects.

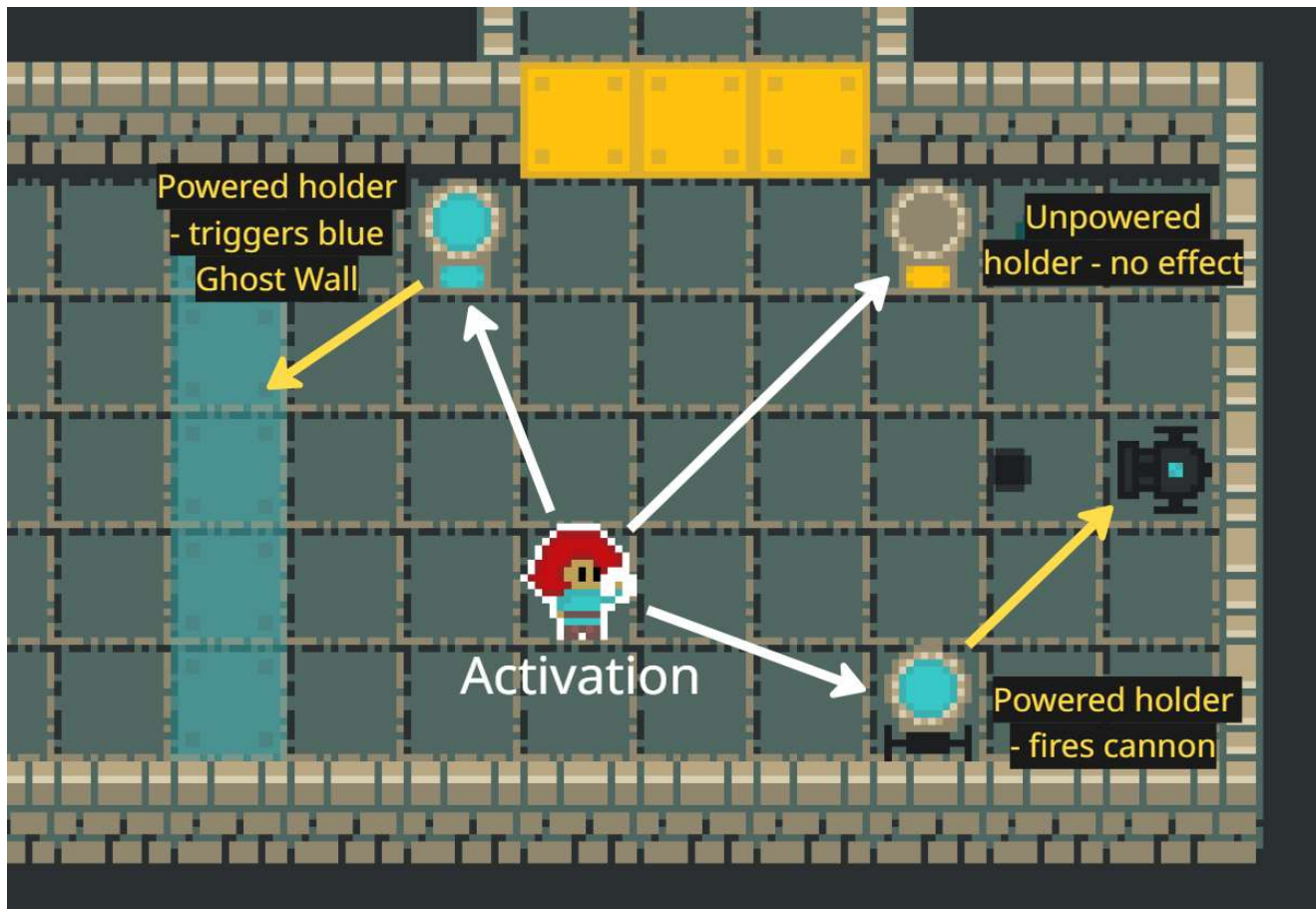


Figure 1 - Annotated screenshot of Echo system

The Echo System can progress the player towards the goal of reaching the end point by unlocking previously closed off areas for example, but it can also deter the player as some powered holders may have unwanted effects. Sub-goals required to complete the main goal include collecting orbs, finding the correct combination of powered and un-powered holders at any given moment, level exploration, and surviving dangerous traps and hostile creatures along the way.

All base game levels were built in the game's custom built level editor. The editor is also included in-game for players to design and create their very own levels. This adds to the replayability of the game and encourages emergent gameplay. All gameplay elements including the tile set, items, and creatures are included for use in the editor, along with a set of tools and options to aid the level creation process. A play mode is included for testing levels during the creation process, and for validating the completed level to ensure winning is possible. Levels are saved in The Gallery, where they can be further edited or played just like the base game levels.

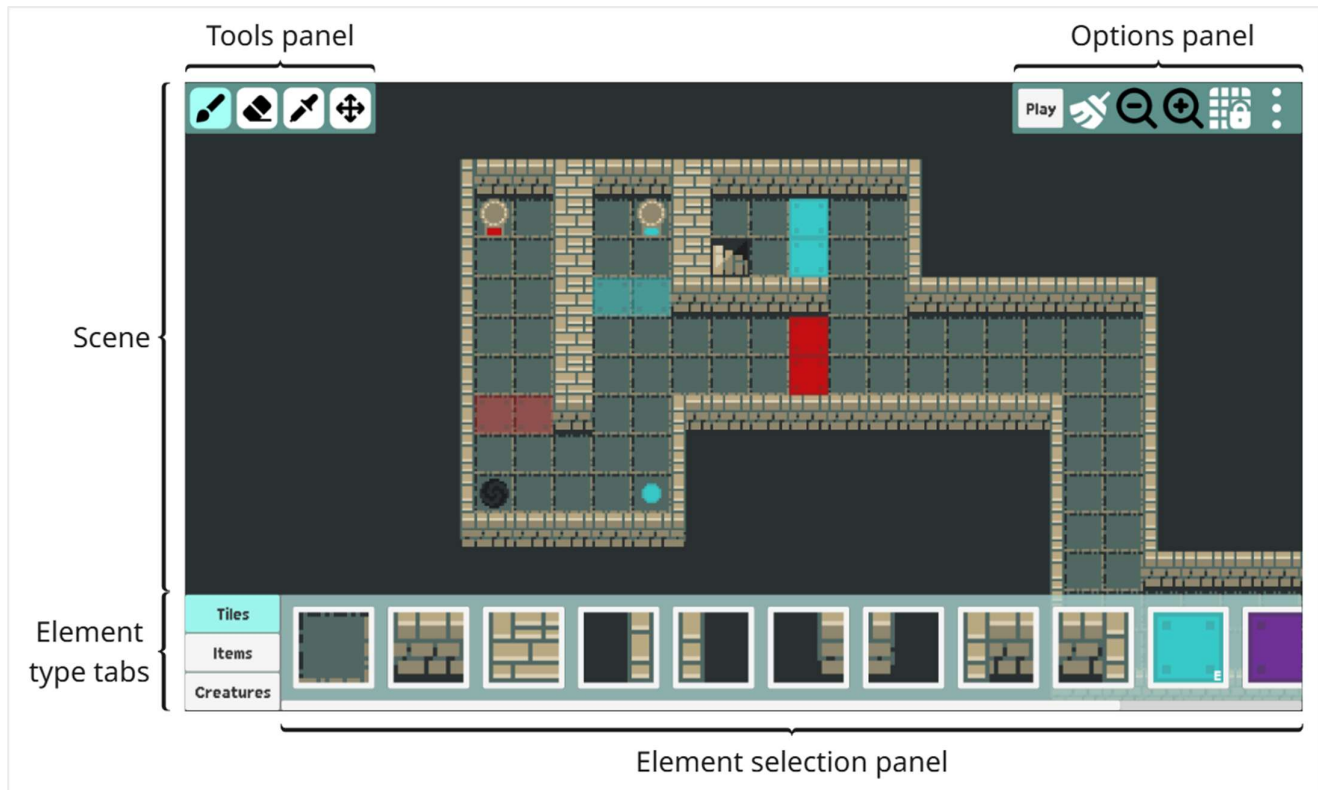


Figure 2 - Annotated screenshot of level editor

Windows was chosen as the target platform for this game, as PC controls are ideal, especially for the level editing features. With PC, level files can also be easily shared with other players. Expansion to other operating systems is a planned feature for the future. Gameplay uses standard keyboard controls for movement, interaction, and Echo activation; and the editor uses mouse controls with some keyboard actions.

Figure 3 shows the flow between UI screens, with each shape representing a screen and each colour representing a scene. The arrows show transitions between the various screens. The main menu includes a settings screen with volume controls for music and sound effects, and a clear progress button. Volume controls can also be accessed from the pause menus in-game. A credits screen, exit button, and navigation to the level selection and gallery scenes are also included in the main menu. Selecting a level in the level selection scene leads to loading the level in the game mode scene. From the gallery the player can load a custom level in play or edit mode and can switch between the two. Game and play mode both feature pause and how to play screens.

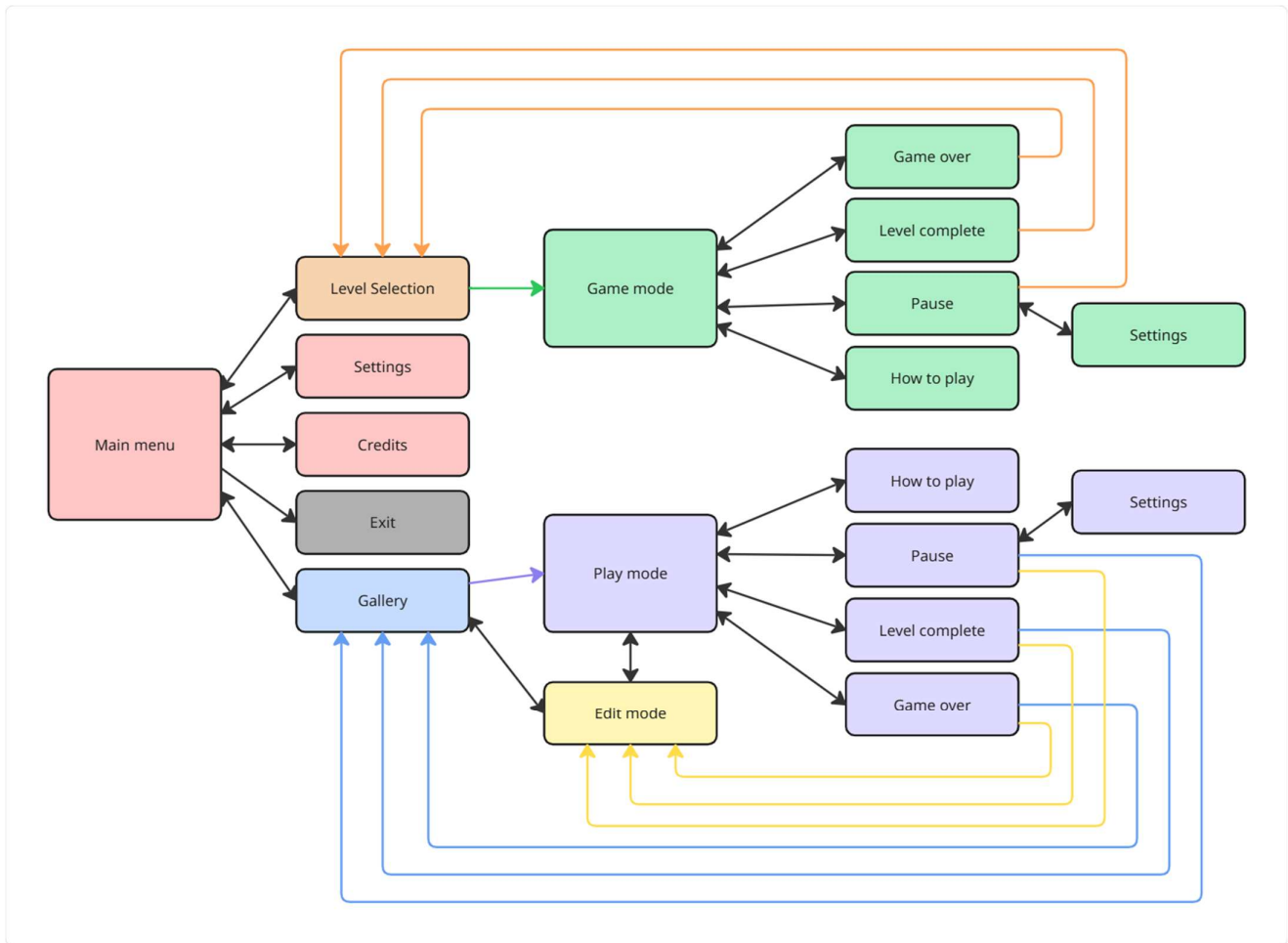


Figure 3 - UI flow

3. Implemented Design and Game Systems

3.1 Core Gameplay Loop

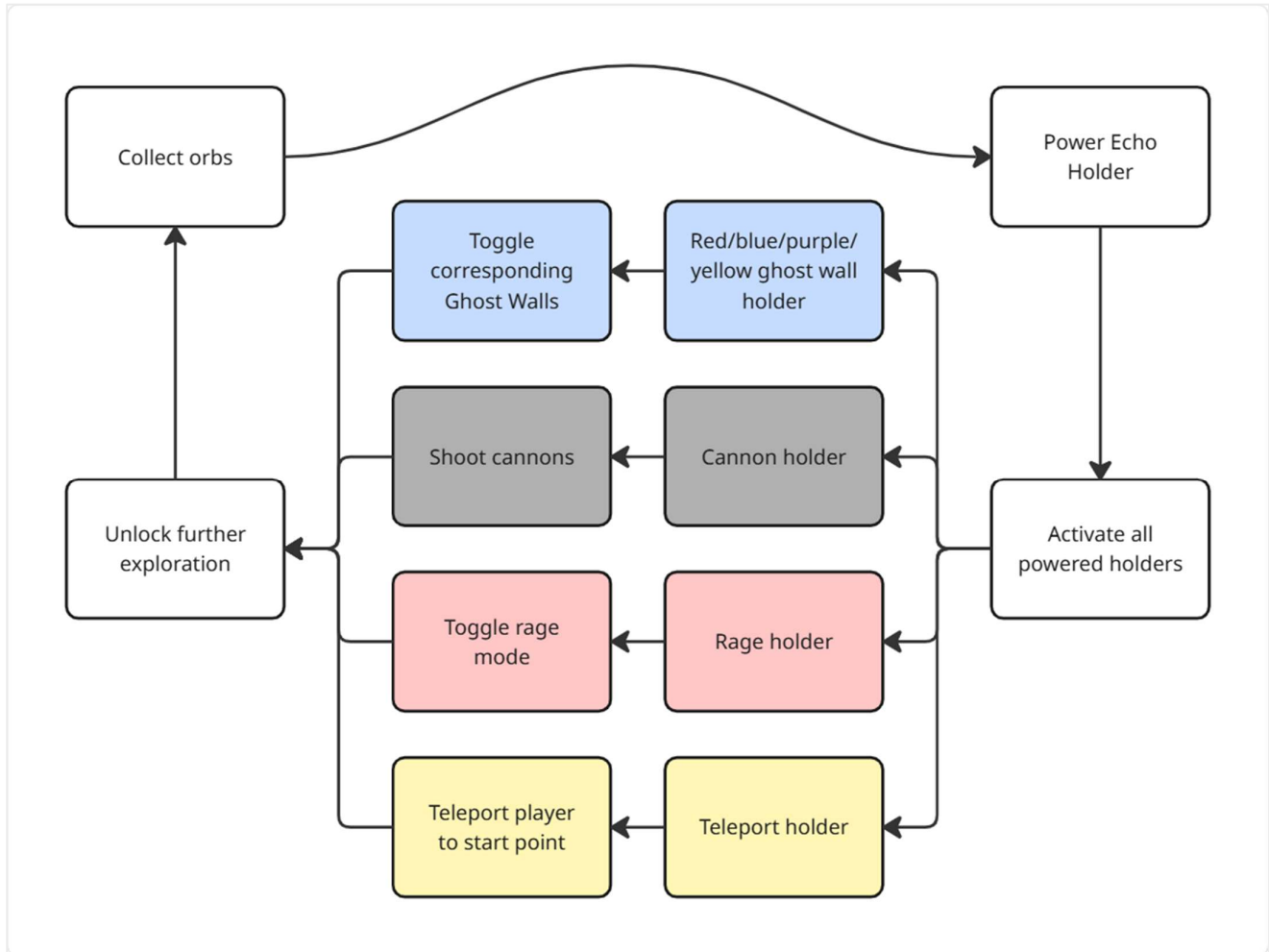


Figure 4 - Core gameplay loop

The core gameplay loop is:

Collect orbs → power Echo Holder → activate all powered holders → unlock further exploration.

As seen in Figure 4, several actions occur between activating all holders and unlocking further exploration. These actions drive player progression within the gameplay loop.

3.1.1 Collect Orbs

Echo Orbs are scattered around the levels. In most levels, players must collect them to progress. The player can hold multiple orbs at once, and the number of orbs the player is currently holding is displayed in the Heads-Up Display (HUD). Orbs may be obtained in three different ways:

- **On the floor:** Orbs may be left lying on the floor which the player can pick up by simply standing close to them.
- **In an Echo Holder:** Sometimes Echo Holders already have an orb inside of them which the player may choose to remove if they are in close range of the Holder.
- **From a Creature:** Some creatures drop an orb upon death. The orb is then left on the floor which may be picked up by the player.

3.1.2 Power Echo Holder

There are various types of Echo Holders, each of which can be in one of two states at any given time:

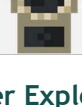
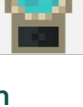
- **Powered:** Has an Echo Orb inside of it.
- **Unpowered:** Empty.

An Echo Holder may be powered/unpowered by placing/removing an Echo Orb. Powered holders respond to Activation events whereas unpowered holders do not.

3.1.3 Activate All Powered Holders

An activation event occurs when the player presses the activate key. The event sends out a signal to all powered holders telling them to perform their assigned action. The player may activate holders from anywhere in a level, regardless of proximity. Table 1 describes the effect of activating each type of holder.

Table 1 - Echo Holder Types

Name	Unpowered	Powered	Activation effect	Related items
Blue Ghost Wall Holder			Toggle Blue Ghost Wall state	Blue Ghost Wall
Red Ghost Wall Holder			Toggle Red Ghost Wall state	Red Ghost Wall
Purple Ghost Wall Holder			Toggle Purple Ghost Wall state	Purple Ghost Wall
Yellow Ghost Wall Holder			Toggle Yellow Ghost Wall state	Yellow Ghost Wall
Cannon Holder			Fire all cannons in level	Cannon
Rage Holder			Toggle Rage Mode, angering all Simple Creatures in the level and all awake Disguised Creatures.	Simple Creature, Disguised Creature
Teleport Holder			Teleport player to the start point.	Start point

3.1.4 Unlock Further Exploration

Activating holders unlocks further exploration by unlocking blocked paths, eliminating enemy creatures, toggling “rage mode”, teleporting the player, or a combination of these effects. From there the player can continue collecting Echo Orbs, completing the gameplay loop.

3.2 Player Mechanics

Echo Orb features a top-down style gameplay, with the camera following the player's movement. The player can move around the levels in eight directions (up, down, left, right, plus diagonals), and collides with walls and other gameplay objects. The player collides with Ghost Walls in the "on" state and can walk through them when they are in the "off" state. Ghost Wall states are toggled through the activation of Ghost Wall Holders, and the state of a Ghost Wall can be determined by its transparency, as Ghost Walls in the "off" state appear slightly more transparent.

The level start point defines where the player begins each level. It also serves as the destination to which the player is teleported when a Teleport Holder is activated. A level is completed when the player reaches the end point. A level may have multiple end points, but it can only have one start point.

The player has a health score which is displayed in the HUD. Health is depleted upon collision with a bullet (from a cannon or Crazy Creature), upon collision with creatures in Rage Mode, and upon being crushed by a trap block (resulting in instant death). Health must be managed carefully, as it cannot be replenished within a level. However, health is fully restored at the beginning of a new level. When health is fully depleted, the game is over, and the player may choose to try again from the beginning.

The player may encounter an altered gameplay condition called "Rage Mode", which increases the level of threat from certain creatures and requires more cautious navigation.

A yellow colour indicates to the player that they may perform an action by pressing the action key. If an Echo Holder can be powered/unpowered a yellow ring appears inside of it, and if the player can exit through the exit point a yellow box appears around it.

Gameplay controls:

- [W], [A], [S], and [D] keys (or the arrow keys) move the player in the up, left, down, and right directions respectively.
- The [E] key is used to place or removes an orb from a holder when in close proximity.
- The [E] key is also used to complete a level when in close proximity to a level end point.
- The [Enter] key is used to activate all holders.

Editor controls:

- [LMB] + move mouse performs a different action based on the selected tool (place, erase, select, eyedropper, move).
- [RMB] + move mouse moves the camera.
- [Q] rotates current placeable item -90 degrees.
- [W] rotates current placeable item 90 degrees.
- [E] swaps between normal and alternative item for the following:
 - Ghost Wall ("on" and "off").
 - Echo Holder (powered and unpowered).
 - Mischievous Creature (creature has orb and does not have orb).

3.3 AI Systems

There are four kinds of non-playable characters (NPCs) in *Echo Orb*, each with their own characteristics and behaviours controlled using behaviour trees. A behaviour tree is a model used to control the decisions an AI agent will make based on certain conditions. *Echo Orb* implements its own behaviour tree model which was based on (Pêcheux, 2021), and additionally it implements its own pathfinding algorithm based on (Code Monkey, 2019a; Code Monkey, 2019b). Behaviour trees were chosen due to their modularity, scalability, and ease of debugging compared to alternative AI approaches such as finite state machines.

The behaviour trees are built up of nodes. Each tree holds a reference to the root node. Each node has a current state - running, success, or failure, and a list of all child nodes. Each node evaluates itself and returns its state. There are two main types of nodes as well as several utility nodes. The main node types are check nodes (represented in the following diagrams in yellow) and task nodes (represented in green). Check nodes check a certain condition and return a state based on whether the condition holds. Task nodes try to complete an action and return its state based on whether it was successfully completed, still running, or failed. Utility nodes include sequence nodes (represented as purple diamonds), selector nodes (represented as orange diamonds), and inverter nodes (represented in red diamonds). Sequence nodes evaluate each of its child nodes in turn until one returns a failure or running state. Selector nodes evaluate each of its children in turn until one returns a success or running state, with nodes at the start of the list getting higher priority than later nodes. Inverter nodes have only one child node and returns its inverse state.

A custom A* pathfinding algorithm was implemented for use in various task nodes such as *Task_GoTo*, *Task_Wander*, and *Task_MoveTowardsPlayer*. A* is a search algorithm that ensures the shortest path between two points. It combines Dijkstra's algorithm with a heuristic estimated cost to guide the search.

3.3.1 Simple Creature

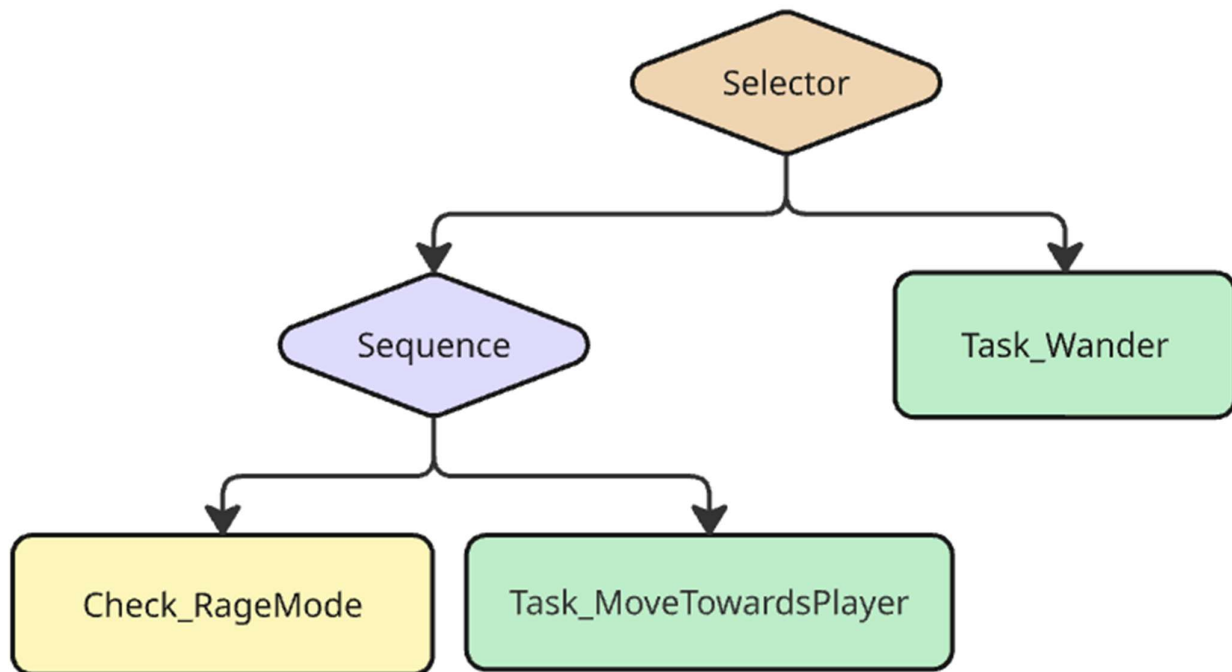


Figure 5 - Simple Creature behaviour tree

The Simple Creature is a normally friendly creature that simply wanders around the levels. However, in rage mode it follows the player and attacks them if they are in range. The Simple Creature does not drop an orb upon death.

3.3.2 Disguised Creature

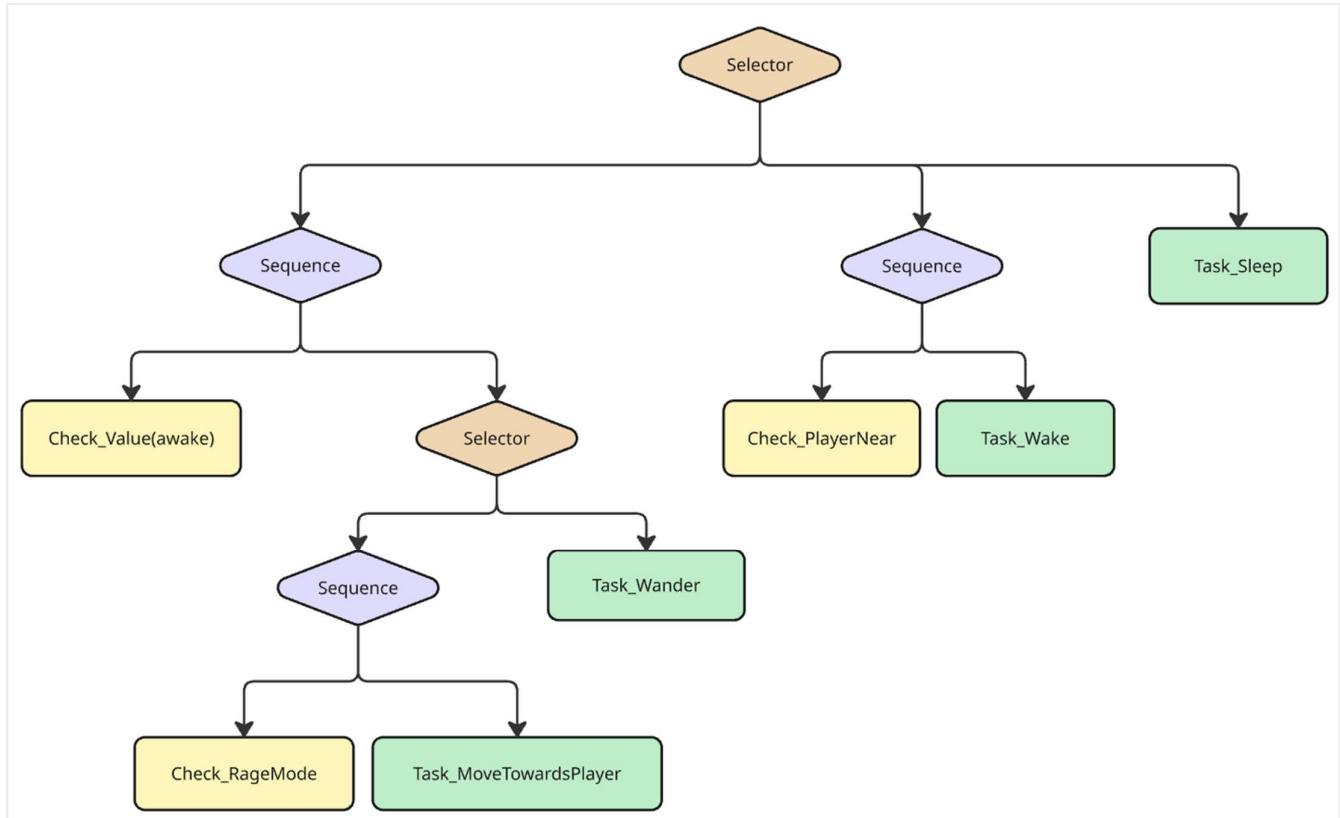


Figure 6 - Disguised Creature behaviour tree

The Disguised Creature starts off asleep, disguised as an orb. When the player gets close it wakes up, revealing itself. Once awake, the Disguised Creature starts acting like the Simple Creature, wandering around the level or chasing and attacking the player if enraged. The Disguised Creature does not drop an orb upon death.

3.3.3 Mischievous Creature

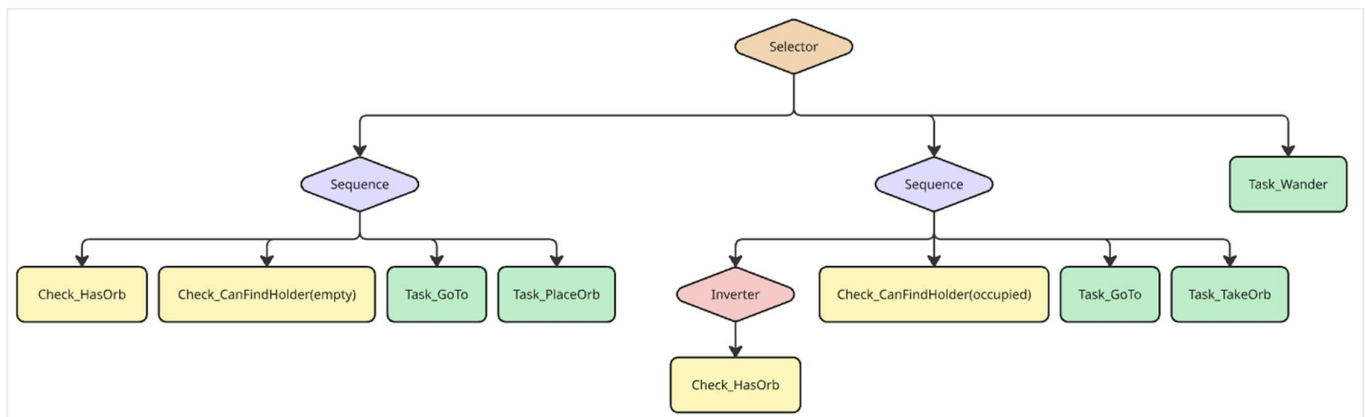


Figure 7 - Mischievous Creature behaviour tree

The Mischievous Creature is non-hostile towards the player. It moves from holder to holder, removing orbs and placing them in different holders. If it cannot find a holder to take/place an orb in, it wanders around the level until one becomes available. The Mischievous Creature drops an Orb upon death only if it is currently holding an orb.

3.3.4 Crazy Creature

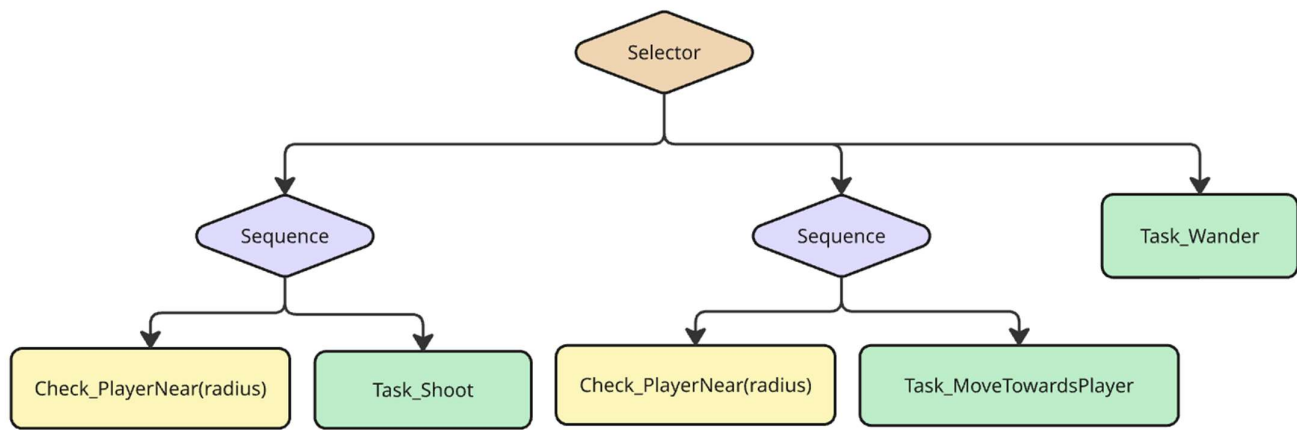


Figure 8 - Crazy Creature behaviour tree

The Crazy Creature is hostile towards the player. It moves towards the player if they are in range, and shoots at them if they are in close proximity. When the player is not nearby it wanders around the level. The Crazy Creature always drops an orb upon death.

3.4 Level Editor

Echo Orb's level editor was not only used to design and create the game's base levels, but is also featured in-game. Players may use the editor to create and play their very own custom levels using existing gameplay elements.

3.4.1 User Interface

The editor displays the level in progress in the middle of the screen and has three main panels used to assemble the level.

Along the bottom of the screen is the element selection panel. This is where all placeable elements can be selected, and are sorted into tiles, items, and creatures categories. When an item is selected, the current tool switches to the place tool.

The panel in the top left-hand corner is the tool selection panel. The currently selected tool is highlighted in blue. The tools include:

- Place: click to place or click and drag to draw elements.
- Erase: click to remove an element or click and drag to remove multiple elements.
- Eyedropper: click on an element to set current placeable item to a matching item and set current tool to the place tool.
- Move: click and drag to move the camera.

The panel on the top right-hand side of the screen contains various other buttons which perform the following actions:

- Play: checks if start and end points have been placed. Missing items are suggested, or if no items are missing, the editor enters play mode, used for testing the created level.
- Clear all: The clear all button displays a dialogue asking if you are sure you want to clear the level. If the player confirms, all placed tiles, items, and creatures are removed from the level.
- Zoom out/in: zooms the camera out or in.
- Snap to grid: toggles grid snapping. Grid snapping is on by default, but when turned off items may be freely placed unrestricted by fixed grid positions.
- Dots: opens the save menu, allowing the player to save, save and quit, or quit without saving.

Elements can be modified when the place tool is selected. The [Q] and [W] key are used to rotate elements left and right respectively (note that tiles cannot be rotated). The [E] key is used to toggle the state of ghost walls and Echo Holders.

3.4.2 Save and load

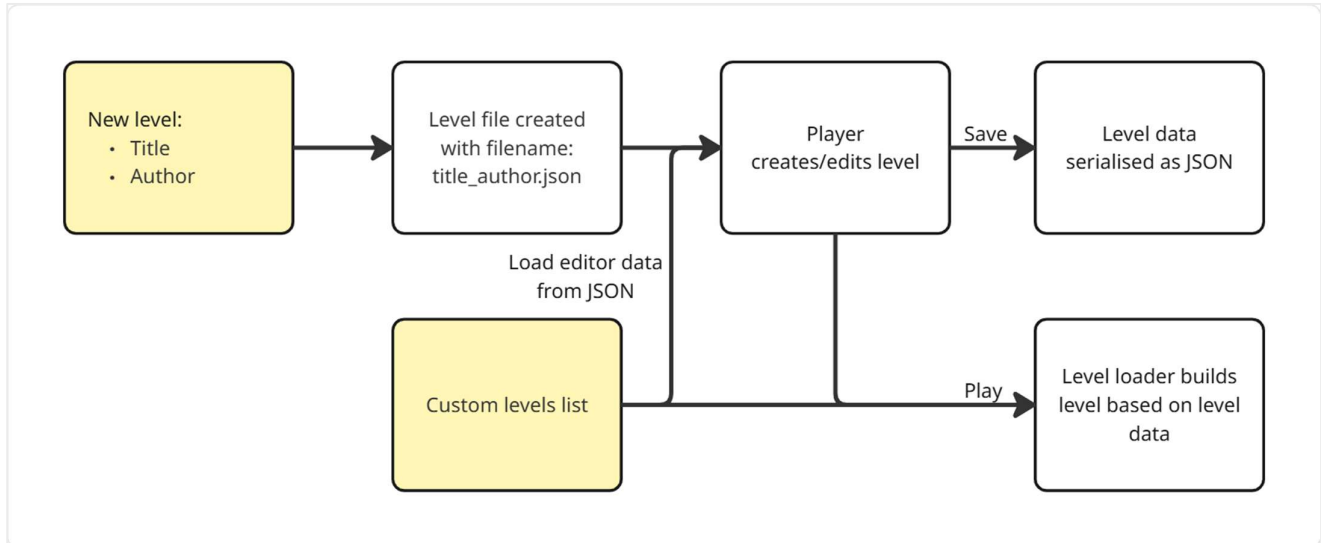


Figure 9 - Save and load

Figure 9 gives a visual representation of the editor’s save/load system. When the player saves their work, the level data is serialised as JSON, and shows up in the custom level list in the gallery with along with the level name and author (both provided by the player upon creation), an incomplete tag if the latest saved version of the level hasn’t been validated, and the date last edited. From here, the player can edit or delete any level, or play one if it is marked as valid. To play a level, a level loader script builds the level from the JSON file. To edit, the same data is instead reconstructed in the level editor.

3.4.3 Level Validation

The editor’s play mode is used for both testing levels during creation and validating finished levels. A level must be validated to be played from the gallery. A level is validated by successfully completing the level in play mode and then selecting the “mark as complete” checkbox. If any further modifications are made to a previously validated level, it is automatically marked as incomplete until it is re-validated.

3.5 Level Design

Echo Orb's base levels were designed to gradually introduce mechanics to the player, while also challenging them to keep them engaged. The first few levels serve to introduce the main mechanics, introducing them, and then combining them to show how they interact with each other. Some levels are more puzzle focused, while others are more action and exploration focused. In general, each levels gets gradually more difficult than the ones before, but there are a few more challenging levels interspersed for variation. Levels are intended to be completed in the listed order but can be played in any desired order to avoid frustration if stuck on a particular level.

Table 2 - Implemented Feature Summary

Feature	Category	Description
Character movement	Gameplay	Eight-directional movement including left, right, up, down, and diagonals.
Player health	Gameplay	Finite health score per level. Decreased upon contact with bullet or enraged creature, or fully depleted when crushed by a block trap. Game over if health score reaches zero.
Follow player camera	Gameplay	Camera configured with Unity's Cinemachine to add smooth following of the player's movement and deadzones where the player can move around slightly in the centre of the screen without the camera changing position.
Echo Orb	Gameplay	Collectable items used to power Echo Holders.
Echo Holders	Gameplay	Fixed position item that can be powered with an Echo Orb. When powered it can be activated. There are multiple different types of Echo Holders, each of which trigger a different effect upon activation.
Ghost walls	Gameplay	Toggleable walls that have two states. When turned on, they act as solid walls. When turned off, they are slightly transparent and may be walked through.
Cannon	Gameplay	Fixed position item that shoots projectiles upon activation of a Cannon Holder.
Rage mode	Gameplay	Causes normally friendly creatures to become hostile towards the player. Simple Creatures and awake Disguised Creatures move towards the players position and attack when in close range.
Ghost wall holder	Gameplay, Echo Holder	Toggles all ghost walls of corresponding colour on activation.
Cannon holder	Gameplay, Echo Holder	Shoots all cannons on activation.
Rage holder	Gameplay, Echo Holder	Toggles Rage Mode on activation.
Teleport holder	Gameplay, Echo Holder	Teleports the player to the start position on activation.
Simple Creature	Gameplay, Creature	Wanders around the level. Becomes hostile in rage mode.
Disguised Creature	Gameplay, Creature	Disguised to look very similar to an orb. When the player is in range it wakes up and starts behaving like a Simple Creature.
Mischievous Creature	Gameplay, Creature	Takes orbs from Echo Holders and moves them to other Holders.

Crazy Creature	Gameplay , Creature	Follows the player if they are near, and shoots at player when in range. Drops an orb on death.
Trap block	Gameplay	Dangerous trap that moves horizontally or vertically at high speed when the player is detected in its line of sight. It moves until it hits a solid obstacle and may crush the player causing instant death.
Start point	Gameplay	The point where the player begins each level, and the point that the player is teleported to on activation of a Teleport Holder. Each level can only have one start point.
End point	Gameplay	A point that the player must reach in order to complete a level. Levels must have at least one end point.
Snap to grid	Editor	Toggles grid snapping. When turned on item placement is restricted by fixed grid positions, when turned off they may be played freely.
Place tool	Editor , Tool	Used to place individual or draw multiple elements at once.
Erase tool	Editor , Tool	Used to erase placed elements.
Eyedropper tool	Editor , Tool	Used to set the current placeable item to a selected item.
Move tool	Editor , Tool	Used to move the camera.
Edit mode	Editor	Environment for building levels.
Test mode	Editor	Environment for testing and validating custom levels.
Move camera	Editor	Camera can be moved with the move tool or by holding the right mouse button.
Zoom camera	Editor	Camera can be zoomed in/out with zoom buttons.
Level validation	Editor	Levels validated by completing them in test mode. Levels must be validated to be played from the gallery.
Rotation	Editor	Items and creatures may be rotated face one of four directions.
Save custom level to file	Save/load	Custom level data saved to file as JSON.
Load custom level from file	Save/load	JSON converted to playable level or to editor data.
Edit custom level	Save/load	Edit/update custom levels.
Delete custom level	Save/load	Deletes level file.
Level info	Save/load	Title, author, date, completed, camera position (in the editor), camera zoom, list of placed items (with ID, x position, y position, rotation), and list of placed tiles (with ID, x position, and y position).

4. Production Process and Project Management

The project was managed using a scrum-lite approach focused on iterative development and incremental task completion. During the planning stage of the project, I created a product backlog outlining all tasks to be completed throughout the development period and updated it as plans evolved. The development period was split into two-week sprints and one three-week sprint, each beginning with a meeting with my supervisor to discuss progress updates, review product backlog, and update sprint backlog. Meetings were conducted online over Zoom. The remainder of the sprint would be spent on implementation and testing of the sprint backlog. Trello was used to manage product backlog, sprint backlog, tasks in progress, completed tasks per sprint, bugs/issues list, stretch features list, and individual task checklists and deadlines.

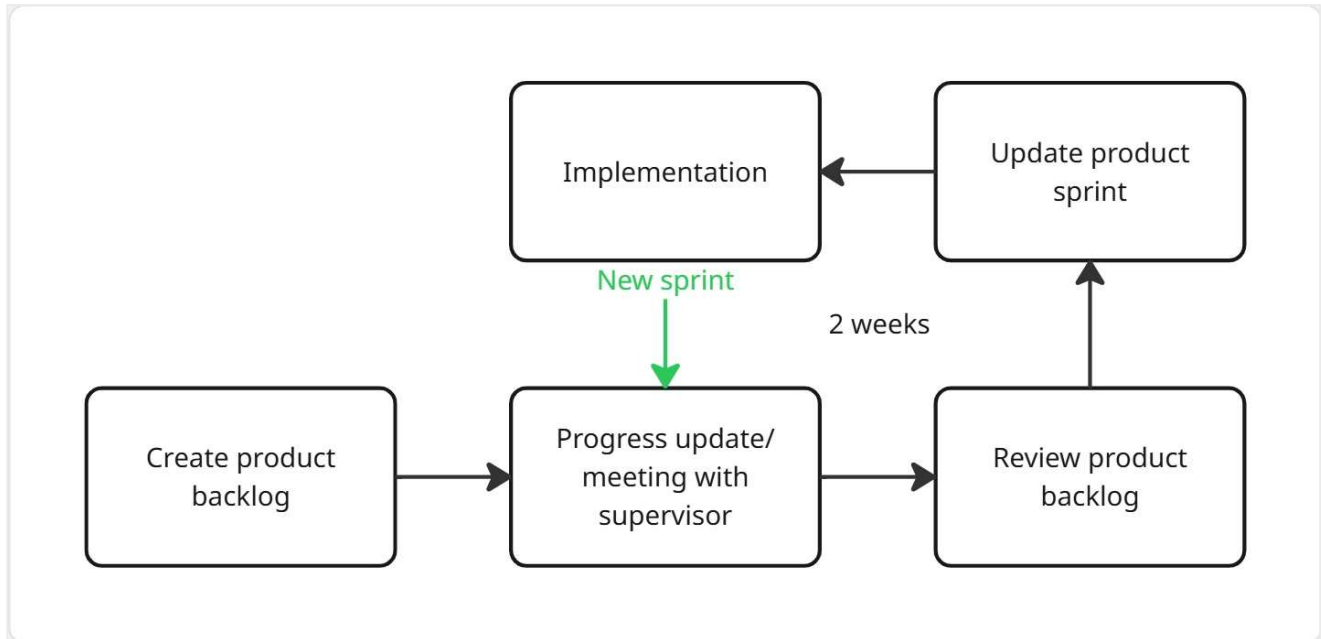


Figure 10 - Sprint overview

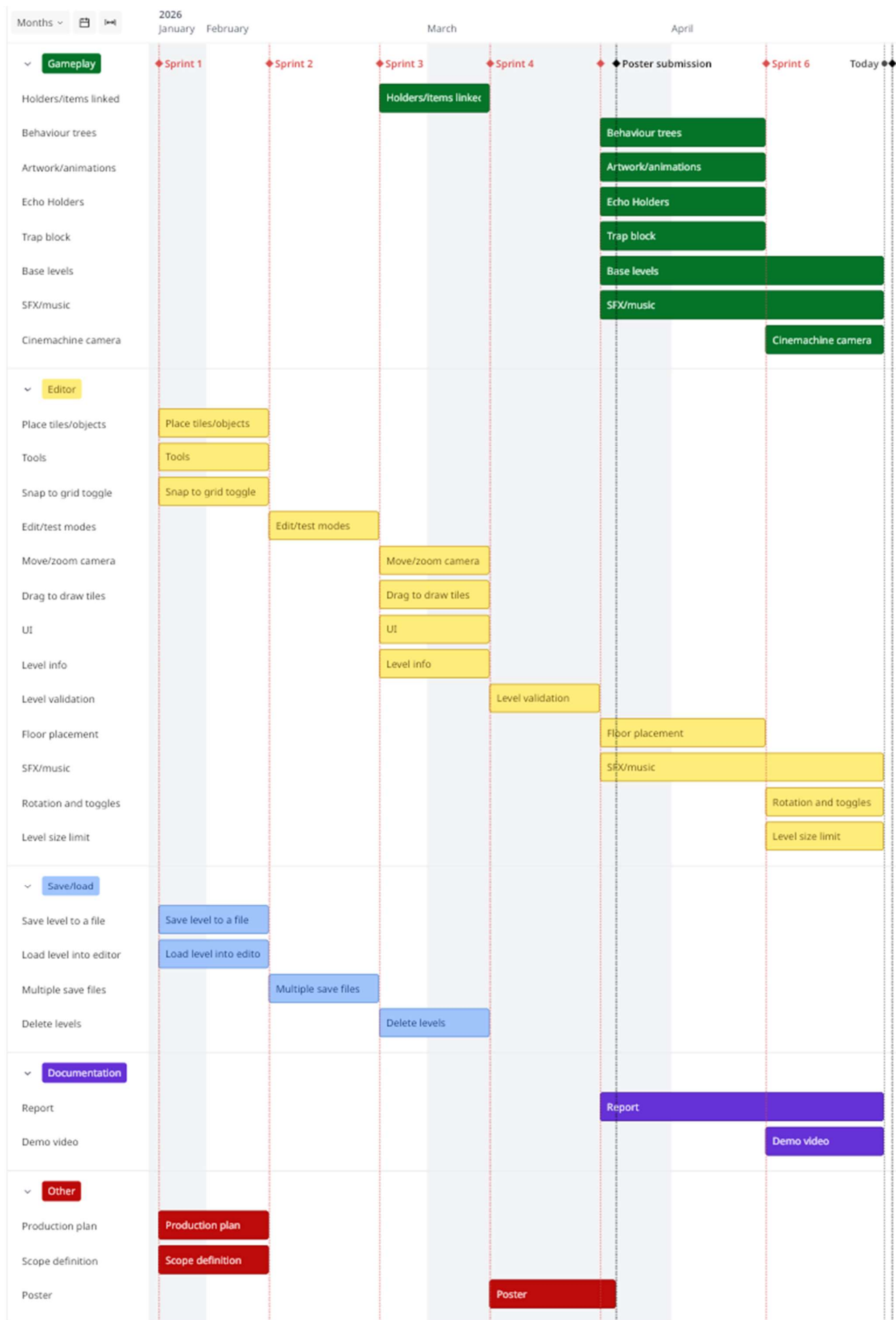


Figure 11 - Gantt Chart

At the very beginning of the development period, I created a scope and production plan. I revised my core, secondary, and stretch features list, adjusting it to maintain an ambitious but achievable scope. With the core gameplay loop present in the Semester 1 vertical slice, I wanted to focus my attention on the level editor for the first few sprints to get it up to a usable level, before bringing my focus back to the main gameplay and adding new editor features as required. With this approach in mind, I split all tasks into three main categories - gameplay, editor, and save/load. The tagging system in Trello allowed me to organise and visualise tasks into these categories. This approach worked well for me as it allowed me to get the level editor up and running early, allowing me to easily create levels and level prototypes using the editor in game allowing me to then test new gameplay features as they were developed. The Gantt chart (see Figure 11) shows how I followed this approach, with editor feature development spanning the entirety of the development period, and gameplay development mainly in the later stages. Save/load features were worked on early. The Gantt diagram also shows task completion and sprint milestones including the beginning of each new sprint, poster submission, and final project submission.

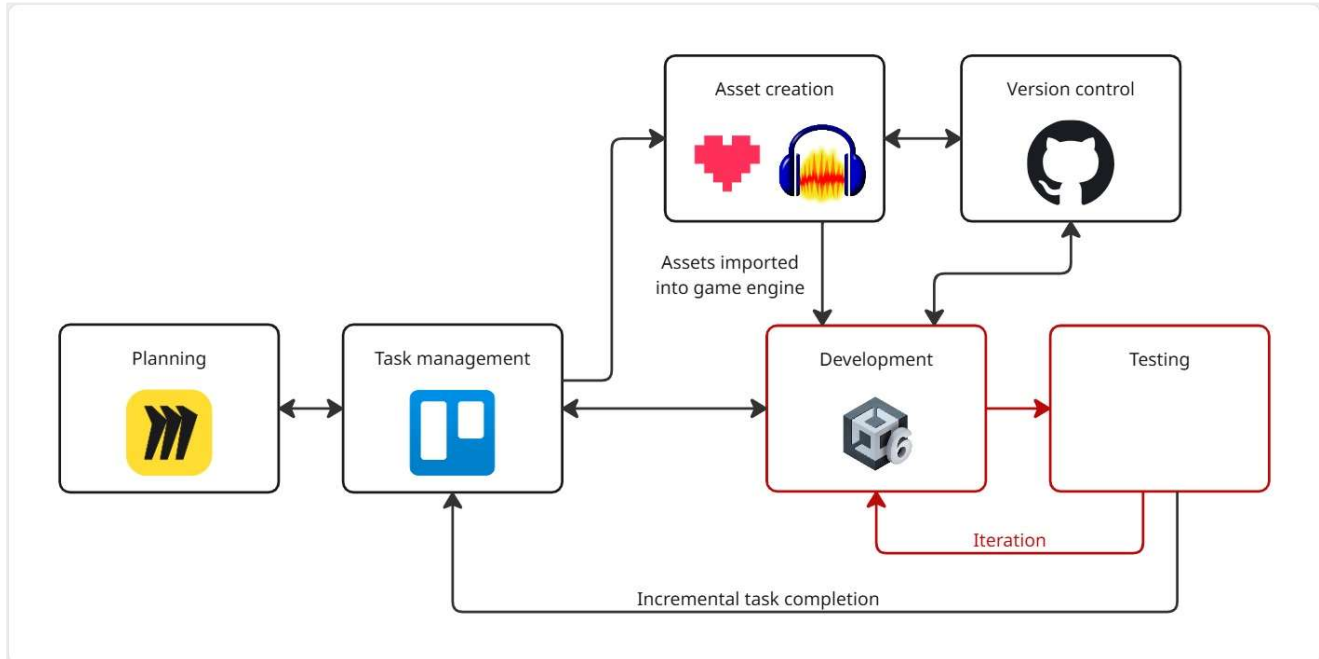


Figure 12 - Development Pipeline Diagram

Figure 12 shows the development pipeline used in the production of *Echo Orb* outlining the overall workflow. It starts with planning and task management, both of which were continuously returned to over the course of development. Tasks included asset creation (sprites using Pixilart and sound effects using Audacity) and development in Unity. GitHub was used to manage the project repository and versioning of the code and assets. The iteration loop (outlined in red in Figure 12) represents the continuous cycle of development and testing. Features were implemented, evaluated, and iteratively built upon and refined based on testing feedback. Table 3 provides more detail on how the tools were used.

Table 3 - Tool usage overview

Tool	Icon	Purpose	Usage
Miro		Planning and diagram creation	Miro was used for creating diagrams which helped visualise complex systems in the planning stages. For example, behaviour tree diagrams were drawn up in Miro prior to implementation (see Figures 5, 6, 7, and 8).
Trello		Task management	Product backlog, sprint backlog, tasks in progress, completed tasks per sprint, bugs/issues list, fixed list, stretch features

			list, and individual task checklists and deadlines managed with Trello.
Unity		Game engine	Unity was used to implement gameplay features and build final game.
Pixilart		Sprite and artwork creation	Created original pixel art sprites to be imported into Unity. This tool was chosen due to its rich feature set and focus on pixel art creation which fit the style of <i>Echo Orb</i> .
Audacity		SFX creation	Recordings from my phone were imported into Audacity to be cleaned and edited to create original sound effects to be imported into Unity.
GitHub		Version control	The project repository is managed on GitHub. Frequent commits along with major releases tagged aligned with the iterative development approach taken.

5. Playtesting and Iteration

My playtesting approach included frequent feature specific short playtesting sessions throughout the development period as well as longer thorough playtesting sessions with a few people closer to the end of the development period.

I began the shorter sessions in late February and held one roughly every two weeks with one of my classmates. The sessions took between 15 and 25 minutes each and consisted of the player testing a recent build and giving feedback about what they liked, disliked, and felt could be changed or improved. Sometimes the sessions were focused on a specific feature that was just implemented or in development. These sessions allowed me to identify bugs, potential issues, and things that I had overlooked.

The longer playtesting sessions occurred mid-April and were held with two different people taking two hours each. I began these sessions in early April. These sessions included a full playthrough of the base game from start to finish, and in-depth testing of the level editor. During these sessions I would observe the players' actions and note what they found easy, difficult, or frustrating. I tried not to give too much instruction about what to do, to see if the player could understand the game's mechanics without external explanation. I would also note any bugs or issues encountered. At the end of the session there would be a discussion where I would ask the player a series of questions about the overall playing experience, the controls, the UI, specific game mechanics, etc., and discuss and any additional thoughts or ideas from me or the player.

Table 4 - Playtesting metadata

Session	Type	Date	Location	Player	Build Version	Duration
1	Short	23/02/2026	SETU	Player 1	V0.1.2	15 minutes
2	Short	09/03/2026	SETU	Player 1	V0.1.3	25 minutes
3	Short	23/03/2026	SETU	Player 1	V0.1.4	15 minutes
4	Long	11/04/2026	Player 1's house	Player 1	V0.1.5	2 hours
5	Long	13/04/2026	Online (Discord)	Player 2	V0.1.6	2 hours

Table 5 - Playtesting feedback and responses

Issue Identified	Evidence	Previously known	Severity	Change Implemented
Session 1				
Player doesn't spawn at spawn point.	Player made a custom level and did not spawn at the placed spawn point.	No	High	Fixed code that broke spawning logic.
Levels can be infinitely sized.	Player walked to see how far they could.	Yes	Low	Add border in the level editor and don't allow player to walk out of bounds.
Additional notes: Wanted to see drag to draw tiles (implemented), multiselecting for deletion (not implemented).				
Session 2				
Easy to accidentally exit the editor without saving the level.	Player accidentally deleted their level by returning to the menu without saving.	No	Medium	Add a pop-up menu asking if you want to save before exiting.
Additional notes: Suggested displaying a label over each item in the editor to explain what it is (not implemented).				
Session 3				

No major issues, player enjoyed the latest level editor features.				
Session 4				
Crazy creature can see player through walls.	Crazy creature was shooting at player behind ghost wall.	Yes	High	Check can see player using raycast.
Low contrast between on/off Ghost walls.	Player didn't realise ghost wall was turned off at first.	No	Medium	Increase transparency on ghost walls.
Popup menus stay on screen unless closed by the player.	Player didn't close popup menu.	No	Low	Close popup menus after a set time passes.
Easy to accidentally clear entire level with clear tool.	Player pressed clear tool thinking it was eraser tool.	No	High	Add a dialogue confirming if level should be cleared (see Figure 14).
Simple Creature moves too quickly.	Difficulty with simple creatures in rage mode due to their fast movement.	No	Medium	Decrease Simple Creature movement speed.
Trap block right raycast origin is incorrectly set.	Trap block was occasionally incorrectly identifying when player was on the right side.	No	High	Fix right raycast origin.
Cannon does not have a collider.	Player can walk through cannon.	No	Low	Add collider to cannon.
Additional notes: Suggested adding a new holder type that teleports the player to a player placed checkpoint (not implemented).				
Session 5				
Trap block plays sound effect repeatedly if player stands beside when it cannot move due to lack of surrounding barriers.	Player created a level with no walls and stood beside trap block and sound effect played repeatedly.	No	Medium	Don't play sound effect unless black has moved.
Game over screen blocks the view of the character.	Player didn't know why they died as the game over screen immediately blocked the game view.	No	Low	Move buttons on game over screen so character is still visible.
Not obvious that you can place a floor.	Player made levels without floors.	No	Medium	N/A
Bullets collide with end point.	Player could hide from bullets behind end point.	No	Low	Disable collision between bullets and end points.
Unclear why levels are marked as "incomplete" in the gallery.	Player asked why their level was marked as incomplete.	No	Medium	Add a question mark button for explanation.
Saving a level without making any changes sets complete level as incomplete.	Player pressed "Save and quit" without making any changes and previously "completed" level	No	Low	N/A

	was marked as incomplete.			
Can unintentionally complete the level by standing near the end point before wanting to end the level.	Player accidentally completed a level before they were ready.	No	Medium	Must press key at the end point to complete level.
Player moves too fast.	Player felt their character was moving too fast.	No	Medium	Decrease player move speed.
UI too big.	Player commented that the buttons in the editor take up too much screen space.	No	Low	N/A
SFX volumes poorly mixed.	Player commented that some sound effects are too loud and others too quiet.	Yes	High	Adjust SFX volumes.
Additional notes: Suggested rule tile placement in the editor (not implemented).				



Figure 13 - Playtesting session 4



Figure 14 - Before/after iteration example

6. Technical Challenges and Solutions

6.1 Challenge 1 - Mischievous Creature Unintended Behaviour

After implementing the Mischievous Creature, I later encountered a severe issue that had previously gone unnoticed. The NPC was occasionally pathfinding towards an Echo Holder and then placing or taking an orb from a completely different Holder. I noticed that this most commonly happened when a creature had an orb and all other Echo Holders in the scene (other than the one it just took the orb from) were occupied, with the creature sometimes even pathfinding to occupied holders. At first, it was quite challenging to identify why the issue was occurring. I first reviewed my behaviour tree structure, and then its nodes, particularly nodes *Check_CanFindHolder*, *Task_GoTo*, *Task_PlaceOrb*, and *Task_TakeOrb*. I added debug log messages to node evaluations as well as points in the code related to target holders, and I identified that the issue was with the target holder not properly synced up between nodes. To fix the issue I added checks to see if the target had updated, if the creature had actually reached the target, and if the holder has updated its occupied status.

Overall, I had difficulty identifying issues with my behaviour trees, as my implementation made debugging difficult. A visual representation of trees, nodes, and current node states would have greatly helped with debugging. Instead, I relied heavily on printing log messages which was tedious and time consuming.

6.2 Challenge 2 - Level Data Handling

Correct handling of level data was challenging to implement. I needed to be able to convert editable level elements to and from level data which can be represented as a playable level. When I initially implemented the level editor and started representing levels in this way, some systems broke such as Ghost Wall Holders triggering Ghost Walls, and the player starting at the correct start point. This was because levels were now being assembled at runtime and references were not being properly assigned. I had to refactor old systems and I built a level builder system that assembles the level and assigns all required references at runtime, making a much more robust solution overall.

6.3 Challenge 3 - Editor Layers

I wanted to implement a layers system in the editor to allow the placement of floor tiles underneath other tiles and items. Implementing a system to allow for this was relatively challenging and introduced a few minor bugs. Initially, I set the entire background to the floor texture, but this was not a long-term solution as I wanted a separate background colour to differentiate between the actual floor and the background. My code disallowed for multiple tiles/items to be placed in the same location, so I had to treat floor tiles as a special case, while also disallowing for floor tiles to be stacked. Once I got this working as expected, a few minor issues appeared. Now when using the eyedropper tool, it would sometimes select the floor item and other times the element on top of the floor. I fixed this by making sure the eyedropper tool prioritises non-floor elements. Using the eraser tool erases both the floor and the above element. This has not been resolved, but it could be done by only erasing the element on top if there are more than one element in one place.

Table 6 - Technical challenges summary

Challenge	Category	Description	Solution	Outcome	Lesson learned
Mischievous Creature unintended behaviour	Bug	Creature occasionally placing/taking orbs from different holders than the one it's at.	Add checks to see if the target had updated, if the creature had reached the target, and if the holder has updated its occupied status.	Mischievous Creature only places/takes orbs from the holder it is standing at	An improved debugging system for my behaviour tree implementation would be very useful for identifying bugs and would save a lot of time.
Level data handling	Challenging feature	Converting level data between JSON, editable level data, and playable level representation.	Create a level builder system to represent level data as a playable level and a save/load system for the editor.	Level data can be represented as JSON, as editable data in the level editor, and as a playable game level.	I learned how to handle level data effectively to be visualised in different ways.
Editor layers	Challenging feature	Ability to add floor beneath other tiles/items in the editor.	Treat floor tile as a special case that can be placed under other elements, or that other elements can be placed on top of.	Elements can be placed on top of floor tiles and floor tiles can be placed underneath other elements.	I learned how to create a simple layers system which can be expanded upon later.

7. Reflection and Learning

Over the course of this project, I developed existing knowledge by putting it into practice and gained new skills. I learned a lot about project management and the tools used for planning, organising, and tracking development. I expanded my technical skills in game development and C# programming. I gained a better understanding of game design choices and the importance of playtesting and feedback.

I realised early on that planning would take place not only at the start of development, but would be an ongoing process throughout the development cycle. Setting ambitious yet realistic sprint milestones with the help of my supervisor was important as it kept me on track without becoming overloaded with unrealistic goals. Early on I had many ideas for the various features that I wanted to implement. I had to make critical design decisions on which features I would prioritise to prevent scope creep. Time management overall and within each sprint was critical in order to achieve my goals. Overall, I prioritised tasks well, however, one mistake I made was not building up the collection of base game levels as new features got added. I made only a small number of base game levels early on and left a large amount of level design until the end of the development cycle. This was problematic as most of the base levels did not get properly playtested, and most of the playtesting that did take place only looked at a small portion of the final set of base levels.

My technical skills were also developed. I had previously learned about concepts such as behaviour trees and A* pathfinding, but I had not put them into practice before. Other elements such as designing a level creation tool and serialising/de-serialising complex level data as a single file were completely new to me and required research and experimentation. Implementing such features and systems involved challenges related to debugging, problem solving, and efficient system design. As all of the game's artwork and animations were created by me, I developed my pixel art design and animation skills.

My semester 1 research focused on emergent gameplay, human-computer interaction, and user generated content. The research I did on these topics helped shape the project and influenced many of the design decisions. I designed the puzzle mechanics and level editor with emergent gameplay in mind. I wanted the Echo system to be modular to allow for player experimentation both in the main gameplay, but also in custom built levels, where new combinations of Echo Holder activations could be played with to create new and interesting puzzle scenarios, and puzzle solutions that I hadn't even thought of. My study of human-computer interaction informed my UI and UX design decisions. I applied some of Nielsen's Usability Heuristics (Nielsen, 1994) and Shneiderman's Eight Golden Rules (Shneiderman and Plaisant, 2010), such as consistent design both across the overall user interface and with similar products, for example using icons that users are already familiar with. Another example is allowing the user to cancel critical actions, such as deleting a level or clearing the scene view in the editor. Playtesting evidence suggests that the editor is intuitive and easy to use, as players had little to no problems understanding how to use it. User generated content is an integral part of the game and level editor, however, user generated content cannot be shared in-game, as no sharing system was implemented. Players can, however, share level files from one game to another, but this unfortunately is not managed in-game, as a feature like this was outside of the project's scope.

The playtesting sessions that I held were very useful, as it allowed me to identify multiple bugs and potential issues, and receive overall feedback and feature suggestions, as discussed in Section 5. One mistake I made, however, was not doing more playtesting with different people. I only had two people playtest the game, and both had similar levels of prior gaming experience. I had planned on playtesting with a larger number of people with different video gaming skill levels. I had no players with limited exposure to video games, which is an issue as I wanted this game to be designed to be accessible for both new and experienced gamers. Testing with a wider range of player backgrounds would have helped ensure that no assumptions were made about unintuitive features for new players, while also avoiding gameplay that is too simplistic for more experienced players.

8. Future Work and Unimplemented Features

Table 7 details unimplemented features which are missing due to a variety of different reasons such as being outside of the project scope and time constraints. Some of these features may be implemented in future versions of the game.

Table 7 - Unimplemented features

Feature	Description	Reason not implemented	Future feasibility
Sharing system	An in-game showcase of player created levels, where levels can be downloaded, played, and rated.	This feature was considered in the very early stages of planning but was well outside the scope of this project.	Feasible
Editor shortcuts	Keyboard shortcuts that would perform common actions such as switching between tools. This would greatly improve the user experience.	This feature was planned but was left too late and insufficient time was left for implementation.	Feasible
Level sorting in gallery	Allow for levels to be sorted alphabetically by title, author, or date modified.	This feature was planned but was left too late and insufficient time was left for implementation.	Feasible
Custom level renaming	Allow players to rename custom levels.	Insufficient time.	Feasible
Editor tutorial	A short tutorial that the player could take to explain how to use the editor.	Unnecessary as editor is already intuitive to use.	Unlikely to implement
Music selector in editor	Allow player to choose a music track from a list of options to be played during that level.	Insufficient time.	Feasible
Undo/redo actions in editor	Actions can be undone and redone using a stack of performed actions.	This feature was started but was more complex than I initially realised and so was cut due to timer constraints.	Feasible
Select to edit elements in the editor	Various actions can be performed on an already placed element, such as moving it, rotating it, toggling alternative variants, etc.	This feature was started but never got completed due to time constraints, and as such was excluded from the finished project.	Feasible
Storytelling and dialogue	Narrative and storytelling elements included with dialogue and a backstory to be discovered.	Insufficient time.	Feasible
Collectable power-ups	Collectable items that give the player new abilities such as increased speed, health, etc.	Does not fit the style of gameplay implemented.	Unlikely to implement
Dynamic “rule-tile” placement in the editor	Correct tiles would be automatically and dynamically placed, for example a corner wall tile is automatically placed at the corner of a wall.	Feature is outside of the project scope.	May be implemented
Descriptions for placeable elements in the editor	When hovering over an element in the editor a short description of the element appears on screen.	This feature was planned but got cut due to time constraints.	Feasible
More interesting behaviour for Disguised Creature	Disguised Creature has more interesting behaviour and characteristics.	This was not a planned feature, but playtesting feedback suggested that this creature was uninteresting and too similar to the Simple Creature.	May be implemented

9. Conclusion

The project resulted in a feature-complete and technically complex game, with room for future expansion. Overall, the project met its original aims. Agile methodology was used to plan, manage, and develop a fully functional game with integrated level editor. The planned core gameplay loop and Echo system are present, with a rich set of gameplay and level editor features. Original assets were created, including all sprites and animations, and a few original sound effects.

Several valuable lessons were learned throughout the development process, and I developed my technical knowledge and skills. The importance of playtesting became evident to me, and I have reflected on how I could improve my playtesting methodology in the future. I have improved my planning and time management skills, as these were used continuously throughout the development of the project. I learned how to apply theoretical knowledge in a practical way by implementing complex systems such as behaviour trees and A* pathfinding, and by applying my semester 1 research in practice. I dealt with difficult technical challenge and had to test and debug critical issues. A few mistakes were made along the way, including leaving the level creation until a late stage of the project, and not doing as much playtesting as I had planned. In future projects, I will address these issues by distributing level design more evenly throughout the development process and conducting more thorough playtesting with a larger group of people.

Overall, this project provided valuable insight into the game development process, and I hope to take what I have learned and apply it to future projects. Thanks to Brendan Lyng for his guidance and support throughout this project.

10. References

Code Monkey (2019a) *Grid System in Unity (Heatmap, Pathfinding, Building Area)*, 4 October. Available at: <https://unitycodemonkey.com/video.php?v=waEsGu--9P8> (Accessed: 28 April 2026).

Code Monkey (2019b) *A* Pathfinding in Unity*, 20 October. Available at: <https://unitycodemonkey.com/video.php?v=aLU04hvz6L4> (Accessed: 28 April 2026).

Nielsen, J. (1994). *Usability Engineering*. Amsterdam: Morgan Kaufmann.

Pêcheux, M. (2021) *How to create a simple behaviour tree in Unity/C#*, Medium, 2 March. Available at: <https://medium.com/geekculture/how-to-create-a-simple-behaviour-tree-in-unity-c-3964c84c060e> (Accessed: 28 April 2026).

Shneiderman, B. and Plaisant, C. (2010) *Designing the User Interface: Strategies for Effective Human-Computer Interaction*. 5th ed. Pearson.

11. Credits

Music

Gameplay music from genxbeats on Pixabay: <https://pixabay.com/music/beats-dark-dungeon-game-hiphop-20241015-252251/>

Menu music from genxbeats on Pixabay: <https://pixabay.com/music/beats-dark-hiphop-20250121-360664/>

Editor music from DeusLower on Pixabay: <https://pixabay.com/music/ambient-atmosphere-dark-fantasy-dungeon-synthpiano-verse-248215/>

Sound effects

Activation sound from Leszek_Szary on Pixabay: <https://pixabay.com/sound-effects/teleport-14639/>

Place Orb sound from Leszek_Szary on Pixabay: <https://pixabay.com/sound-effects/scale-c7-106132/>

Take Orb sound from Leszek_Szary on Pixabay: <https://pixabay.com/sound-effects/scale-g6-89174/>

Player damage sound from cabled_mess on Pixabay: <https://pixabay.com/sound-effects/hurt-c-08-102842/>

Pickup Orb sound from twohoursago on Pixabay: <https://pixabay.com/sound-effects/glass-bottle-clink-90671/>

UI Pop sound from Pixabay: <https://pixabay.com/sound-effects/film-special-effects-ui-pop-sound-316482/>

Other

Font from Catterio Sylt on dafont.com: <https://www.dafont.com/ari-w9500.font?text=Echo+Orb+A&back=bitmap>

Icons from www.flaticon.com